OpsPilot

Product Requirements Document — Agentic AI Operations Assistant for retail operations

Take-home submission for the Forward Deployed Engineer role · Rishabh Kumar Raizada

Live app: https://92-4-93-133.sslip.io · Repository: github.com/RishabhRaizada/agentic-ops-assistant · Demo password for every role: Demo@2026 · Version 1.1, 3 Nov 2026

Approach. I treated the brief as a customer engagement: start from how the operations team works today (email, spreadsheets, chasing approvers), automate the legwork, keep authority and money with the people and rules the business already trusts, and design every integration point so it can be swapped for the customer's real systems during deployment. Section 9 is the deployment plan I would run with this customer.

1. Problem and target users

A growing retail chain runs its operational exceptions — delayed deliveries, stock shortages, supplier problems, urgent spend approvals and customer commitments — through email, spreadsheets and personal follow-up. Each request is read by a person, who gathers data from several systems, judges the impact, chases stakeholders and writes updates for management. The result is slow decisions on time-critical issues, no single view of what is open or overdue, approvals that depend on who happens to be online, and no reliable record of who decided what and why.

The assignment's sample (500 display units for a Pune store launch delayed by 10 days, 120 in the warehouse, approval needed by 4 PM) is typical: the facts are scattered, the deadline is hours away, the approver depends on the amount, and the cost of a wrong call is a delayed store opening.

User	Job to be done	What OpsPilot gives them
Ops analyst	Triage requests, find the facts, propose a fix	Extracted, system-checked facts; costed options; one-click routing
Ops manager / Regional head	Decide quickly within their authority	Approval inbox with rationale, cost working, SLA timer; reject-with-constraint
Director / Finance	Large spend decisions, portfolio oversight	Escalations, management summaries, risk and revenue-at-risk view
Store manager / Supplier / Requester	Know what is happening	Accurate, human-approved communications
Auditor	Verify decisions were made correctly	Append-only, hash-chained trail of every AI, human and system action

2. Proposed solution and workflow

OpsPilot is a supervised multi-agent system: specialised LLM agents read, interpret and propose; deterministic code computes, verifies, authorises and routes; people decide. Each request becomes a case that moves through an explicit state machine to closure.

Receive (email, manual entry, .eml upload or signed webhook) → **Screen** (prompt-injection guard) → **Extract** (facts with evidence quotes) → **Correlate** (duplicate / update / new) → **Verify** (inventory, PO, store calendar) → **Assess risk** (agent + rules engine) → **Plan** (tool-using planner + reviewer) → **Human review & approval** (policy routing, SLA, escalation) → **Execute & communicate** → **Close** (resolution reason, management brief).

Lifecycle: NEW → SCREENING → ANALYZING → NEEDS_INFO | NEEDS_REVIEW | READY_FOR_APPROVAL → AWAITING_APPROVAL → APPROVED → IN_PROGRESS → RESOLVED → CLOSED, with REPLANNING (after rejection), ON_HOLD (material change), MERGED (duplicate) and CANCELLED. Illegal transitions are rejected; every transition is an audit event.

3. Functional requirements

#	Requirement	How it is met
F1	Read an email or manual request	Simulated ops mailbox, manual form, .eml upload, HMAC-signed inbound webhook
F2	Extract information, deadlines, dependencies	Intake agent returns typed fields with status, confidence and an exact evidence quote; code normalises numbers, money and relative dates (“4 PM today”, “next week”) in IST
F3	Assess urgency, impact, risk	Risk agent (urgency P1–P4, impact, revenue at risk) cross-checked by an independent rules engine
F4	Identify missing / conflicting information	Required-field rules per request type; reconciliation of email claims against inventory, POs and store calendar
F5	Recommend actions with rationale	Planner agent queries read-only tools and proposes 2–3 costed options citing tool results; Reviewer agent critiques (≤1 revise loop)

#	Requirement	How it is met
F6	Route for approval or escalation	Policy engine picks the approver by amount, region and limit; SLA timer, reminder, automatic escalation up the chain
F7	Review and override AI output	Edit any fact with a reason (AI value kept), choose a different option, edit cost, or write an operator option
F8	Track until closure	State machine, action checklist, per-state timestamps, resolution reasons, dashboard and inbox filters
F9	Stakeholder comms and management summaries	Comms agent drafts per audience from verified facts; human clicks send; Reporter agent writes case briefs and portfolio digests from computed metrics
F10	Audit trail of AI and human actions	Append-only, SHA-256 hash-chained events (DB triggers block edits); per-agent run traces with prompt version, model, inputs, outputs

4. Non-functional requirements

- **Security:** role-based access with region scope and approval limits enforced server-side; separation of duties (no self-approval); httpOnly session cookie + CSRF header; HTTPS; prompt-injection defence; secrets only in environment.
- **Reliability:** five-model fallback chain across two providers → deterministic rules; circuit breakers on integrations; outbox with retry and dead-letter; idempotent approval decisions; the pipeline never leaves a case stuck.
- **Auditability & observability:** every agent run and human action recorded; chain verification endpoint; health/readiness probes; AI-quality metrics (override, fallback, self-correction, ungrounded-extraction rates).
- **Performance & cost:** first full analysis in about 25–35 s (UI shows each agent step live); page loads under 1 s; around ₹1–2 of model usage per request.
- **Usability:** every AI value shows its source and confidence; evidence is highlighted in the original email; plain-language explanations for every block or refusal.

5. Agentic AI and LLM design

Agent	Responsibility	Guardrails
Guard	Classify inbound text as safe / suspicious / malicious	Sees datamarked text only; combined with regex heuristics
Intake	Split multi-issue emails; extract fields, deadlines, dependencies, stakeholders	Quotes must be exact substrings of the email (validator forces retry)
Correlation	Duplicate vs update vs new, given code-retrieved candidates	Must name a real candidate; auto-merge only at ≥ 0.85 confidence
Risk & impact	Urgency, risk level, business impact with fact citations	Unknown fact refs rejected; compared with rules engine
Planner	Query inventory, transfers, alternate suppliers, expedite quotes, POs; propose costed options	Read-only tools; costs must cite tool results; budget caps and rejected options enforced by validator
Reviewer	Critique the plan against facts and tool results	Can force one revision or block the plan
Comms / Reporter	Stakeholder drafts; management narrative	Verified facts and pre-computed numbers only; never sent without a human

How agents coordinate and share context

Agents never call each other. The orchestrator (plain Python state machine) builds a minimal context pack for each agent from the case's fact ledger — every fact has a reference (F1…Fn), value, source (email, inventory, PO, calendar, human), status and confidence, with AI and human values stored side by side. Agents must cite fact refs and tool refs (T1…Tn); typed output contracts are validated before anything is stored. A human correction increments the ledger version and re-runs only downstream agents; approvals bound to the old plan hash are withdrawn.

When human approval is mandatory

If any option commits spend, risk is high or critical, a customer commitment or supplier order changes, composite confidence is below 0.75, injection is suspected, or the case ran in degraded mode. The approver is chosen by amount (≤ ₹50k Ops manager, ≤ ₹5L Regional head, above that Director) and must hold authority for the region and amount. Every outbound stakeholder email requires a human click. No agent has an approve, send or spend capability, so authority can never be granted by model output.

How confidence is determined

Self-reported model confidence is only one input. Field confidence = min(model confidence, grounding: 1.0 verbatim quote, 0.6 inferred, 0.3 unsupported, 0 missing). Overall = extraction × agent-vs-rules risk agreement × 0.7 per unresolved conflict × reviewer verdict × citation validity, capped by the planner's own confidence and at 0.55 in degraded mode. Bands: high ≥ 0.75, medium, low < 0.5 (AI option cannot be submitted without a written justification). The breakdown is shown in the UI.

How unsupported or incorrect outputs are handled

Schema validation with retries; grounding and citation validators that make the model correct itself; confidence caps; reviewer critique loop; rules-engine cross-check (two-level disagreement becomes a blocking “agents disagree” issue); human override with mandatory reason; fallback model, then deterministic extraction and planning flagged for review. Nothing executes automatically.

How agent actions are monitored and audited

Each agent run stores prompt version, model actually used, fallback flag, input hash, tool calls, validation retries, tokens and latency, visible on the request's timeline. Analytics reports success, fallback, self-correction, human-override and ungrounded rates per agent. The audit log is append-only and hash-chained, with a one-click integrity check and CSV export.

6. Edge-case handling

Scenario	Detection → communication → recovery → record
Missing information	Required-field rules → “Waiting for info”, SLA paused, clarification email drafted → resumes when reply or correction fills the fact
Conflicting information	Email (150) vs inventory (100) → blocking conflict card with both sources → human picks one, recorded as an override, analysis re-runs
Duplicate requests	Content hash or entity + fuzzy retrieval → Correlation agent → merged into the original, requester linked, owner notified
Unclear intent / multiple issues	Intake flags multi-issue → one-click split into linked cases; ambiguous requests get clarification questions
Incorrect AI extraction	Human edits a field with a reason → AI value preserved → downstream agents re-run on the new facts
Low-confidence recommendation	Composite confidence < 0.5 → blocking “insufficient evidence” → justification or operator-defined option required
Approval delay	SLA monitor → reminder 60 min before due → automatic escalation to next level → SLA breach alert to management
Approval rejection	Reason (and optional budget cap) required → Planner re-plans under constraints → new approval routed
Changing circumstances	Supplier revises delay 10 → 6 days → facts updated, pending approval superseded, approver told, case re-assessed
System / LLM unavailable	Fallback model; if all fail, deterministic extraction and planning (flagged). Inventory outage: circuit breaker, figures marked unverified, retry
Unauthorized action	Role, region, limit and self-approval checks in code → refusal with reason → pinned audit event
Prompt injection	Heuristics + Guard on datamarked text → spans highlighted, forced human review; no agent can approve or send
Communication failure	Outbox retries ×3 with backoff → dead-letter, issue raised, owner notified in-app → retry or in-app fallback
Contradictory agent outputs	Risk agent vs rules ≥ 2 levels apart → higher risk taken, blocking acknowledgement required
Request cancellation	Launch postponed → approvals withdrawn, case on hold → on confirmation: cancelled, actions stopped, updates drafted

All 15 are implemented; each has an automated test and a simulated email or admin control for live demonstration.

7. Assumptions and trade-offs

- **Assumptions:** inventory, PO, supplier-quote and store-calendar systems are reachable through APIs (mocked with seeded data and fault injection); amounts in INR; business timezone IST; email arrives through a mailbox webhook or forwarding rule; approvers act in the web app.
- **Deterministic orchestrator instead of an agent graph framework:** approvals wait hours, so they are database rows with due times rather than paused graph threads; every transition is auditable and testable. Less “autonomous” by design.
- **Cost-optimised model chain:** Gemini 3.5 Flash-Lite handles the routine path; Gemini 3 Flash, 3.5 Flash, 3.6 Flash and Groq gpt-oss-120b take over on errors, rate limits or timeouts (each has its own quota pool); deterministic rules are the floor. Lite models are weaker at long reasoning — the Reviewer agent and validators compensate, and any backend can be promoted by configuration.
- **SQLite + single process:** fits the free 1 GB VM and keeps operations trivial; code is dialect-neutral (Postgres via one setting). Horizontal scale needs a job queue.
- **Mock enterprise systems:** make the demo deterministic and failure-injectable; real ERP/WMS adapters plug in behind one module.

8. Success metrics and MVP scope

Metric	Target (pilot)	Measured by
Request-to-decision time for P1/P2	−60% vs manual baseline	Per-state timestamps
Approvals decided within SLA	≥ 95%	Approval due vs decided time
Extraction fields corrected by people	< 10%	Override rate (AI quality panel)
AI option accepted by approver	≥ 60%	Approved option was the recommended one

Metric	Target (pilot)	Measured by
Unauthorized approvals	0	Policy enforcement + audit
Stakeholder messages delivered	≥ 99% (after retry)	Outbox status

MVP (built): intake through closure for the five request types, seven agents, policy-based approvals with escalation, overrides, comms, management reporting, audit, analytics, fault injection and all 15 edge cases.

Roadmap: covered by the deployment plan below, then Postgres + job queue, learning from overrides (few-shot memory and prompt evaluation), calibrated confidence from production outcomes, supplier-performance analytics.

9. Deploying at the customer

How I would take OpsPilot from this prototype to production with the retail operations team, as the engineer embedded with them. Each phase has an exit criterion agreed with the ops lead; nothing moves to the next phase on enthusiasm alone.

Phase	What I do with the customer	Exit criterion
0. Discover (week 0)	Sit with the ops desk; collect 50–100 real requests from the last month; map the approval matrix, regions and limits as they actually work; baseline time-to-decision and missed deadlines	Signed-off workflow map, policy table and baseline metrics
1. Integrate (weeks 1–2)	Connect the ops mailbox (Microsoft Graph or Gmail push → signed webhook); replace mock adapters with read-only ERP/WMS/PO APIs or the existing stock spreadsheets; SSO; load the real approval matrix in Admin; agree data-handling rules	Real requests flow in; adapters pass the existing contract tests
2. Shadow (week 3)	OpsPilot analyses every request while the team works as today; I review extraction and recommendations against what people actually did and tune prompts, rules and thresholds	≥ 90% field accuracy on real mail; no blocking defects
3. Assisted pilot (weeks 4–6)	One region (West) decides inside OpsPilot; weekly review of overrides, rejections and escalations with the ops lead; fixes shipped within the week	Pilot targets in section 8 met for 2 consecutive weeks
4. Scale & hand over	Roll out region by region; Slack/Teams approvals; Postgres + job queue; runbook, alerting and admin training so the customer's team owns it	Customer team operates it without me

Where each integration plugs in: intake → POST /api/intake/webhook; systems of record → backend/app/integrations/systems.py (one adapter per system, circuit-broken); email → integrations/mail.py (SMTP or provider API); models → environment-configured chain; policy → Admin settings, no code change.